BaseClaw Improvement Spec

Document type: Implementation specification for Claude Code **Project:** BaseClaw — Yahoo Fantasy Baseball MCP Server **Repository:** `github.com/jweingardt12/baseclaw` **Date:** March 22, 2026 **Scope:** MCP compliance, README rewrite, setup simplification, tool consolidation, OpenClaw integration, casual-user accessibility

---

How to use this spec

This document is organized into **4 phases** with **42 discrete tasks**. Each task includes the exact files to modify, the change to make, why it matters, and acceptance criteria. Execute phases in order — later phases depend on earlier ones.

Conventions used in this spec:

- `[FILE]` = file path relative to repo root
- `[NEW]` = create this file
- `[MODIFY]` = edit existing file
- `[DELETE]` = remove this file
- `AC:` = acceptance criteria (how to verify the task is done)

Current state summary

Attribute	Current	Target
MCP tool count	123	25–30 per session (all 123 still accessible)
README length	~800 lines, architecture-first	~400 lines visible, casual-first with collapsible developer sections
Setup time (clone → first query)	30+ minutes	Under 10 minutes
MCP client		

configs in README	2 (Claude Code, Claude Desktop)	6+ (add VS Code, Cursor, Cline, Windsurf)
Example prompts in README	4	12+ organized by workflow
Tool naming convention	`yahoo_*` prefix on most	`{domain}_{action}` consistent namespace
OpenClaw workspace files	SKILL.md + AGENTS.md	Full workspace: SOUL.md, AGENTS.md, USER.md, HEARTBEAT.md, MEMORY.md, 6 skill files
Target audience framing	Developer/architect	Casual fantasy baseball player first, developer second

---

Phase 1: Tool consolidation and MCP compliance

Goal: Reduce effective tool count per session from 123 to under 30 without losing any capability. Fix MCP anti-patterns.

Why this is first: 123 tools exceeds Cursor's hard limit (40), degrades Claude's tool selection accuracy past 30 tools, and consumes excessive context window tokens. Nothing else matters if the tools don't work well.

Task 1.1: Implement configurable toolsets

Priority: Critical **Files:**

- `[MODIFY] mcp-apps/server.ts`
- `[NEW] mcp-apps/src/toolsets.ts`
- `[MODIFY] mcp-apps/main.ts`
- `[MODIFY] .env.example`

What to build:

Create a toolset system that filters which tools are registered based on configuration. Define these toolset categories:

```
// mcp-apps/src/toolsets.ts

export const TOOLSETS: Record<string, string[]> = {
  // Core — always loaded (~12 tools)
  core: [
    "yahoo_roster",
    "yahoo_standings",
    "yahoo_my_matchup",
    "yahoo_free_agents",
    "yahoo_search",
    "yahoo_player_stats",
    "yahoo_value",
    "yahoo_compare",
    "yahoo_matchups",
    "yahoo_injury_report",
    "yahoo_rankings",
    "yahoo_player_list",
  ],

  // Lineup management (~6 tools)
  lineup: [
    "yahoo_auto_lineup",
    "yahoo_lineup_optimize",
    "yahoo_set_lineup",
    "yahoo_week_planner",
    "yahoo_game_day_manager",
    "yahoo_roster_health_check",
  ],

  // Waiver & transactions (~8 tools)
  waivers: [
    "yahoo_waiver_recommendations",
    "yahoo_waiver_claim",
    "yahoo_waivers",
    "yahoo_streaming",
    "yahoo_add",
    "yahoo_drop",
    "yahoo_swap",
    "yahoo_transaction_trends",
  ],

  // Trade tools (~6 tools)
  trades: [
    "yahoo_trade_analysis",
    "yahoo_trade_pipeline",
    "yahoo_propose_trade",
```

```
  "yahoo_pending_trades",
  "yahoo_accept_trade",
  "yahoo_reject_trade",
],

// Strategy & analytics (~8 tools)
strategy: [
  "yahoo_category_check",
  "yahoo_matchup_strategy",
  "yahoo_scout_opponent",
  "yahoo_punt_advisor",
  "yahoo_league_landscape",
  "yahoo_power_rankings",
  "yahoo_season_pace",
  "yahoo_playoff_planner",
],

// Workflow aggregators (~8 tools)
workflows: [
  "yahoo_morning_briefing",
  "yahoo_weekly_digest",
  "yahoo_season_checkpoint",
  "yahoo_waiver_deadline_prep",
  "yahoo_whats_new",
  "yahoo_game_day_manager",
  "yahoo_roster_health_check",
  "yahoo_league_landscape",
],

// Intel & Statcast (~10 tools)
intel: [
  "fantasy_player_report",
  "fantasy_regression_candidates",
  "fantasy_reddit_buzz",
  "fantasy_trending_players",
  "fantasy_news_feed",
  "fantasy_transactions",
  "yahoo_statcast_history",
  "yahoo_projection_disagreements",
  "yahoo_zscore_shifts",
  "yahoo_ownership_trends",
],

// Prospects (~8 tools)
prospects: [
  "fantasy_prospect_rankings",
  "fantasy_prospect_report",
```

```
  "fantasy_prospect_compare",
  "fantasy_prospect_buzz",
  "fantasy_prospect_watch",
  "fantasy_callup_wire",
  "fantasy_eta_tracker",
  "fantasy_stash_advisor",
],

// Draft (~6 tools)
draft: [
  "yahoo_draft_recommend",
  "yahoo_draft_cheatsheet",
  "yahoo_best_available",
  "yahoo_draft_results",
  "yahoo_keeper_analysis",
  "yahoo_draft_recap",
],

// MLB reference (~7 tools)
mlb: [
  "mlb_teams",
  "mlb_roster",
  "mlb_player",
  "mlb_stats",
  "mlb_schedule",
  "mlb_injuries",
  "mlb_standings",
],

// League history (~8 tools)
history: [
  "yahoo_league_history",
  "yahoo_record_book",
  "yahoo_past_standings",
  "yahoo_past_draft",
  "yahoo_past_teams",
  "yahoo_past_trades",
  "yahoo_past_matchup",
  "yahoo_rival_history",
],

// Admin / team management (~6 tools)
admin: [
  "yahoo_change_team_name",
  "yahoo_change_team_logo",
  "yahoo_league_context",
  "yahoo_league_pulse",
```

```
    "yahoo_transactions",
    "yahoo_all_rostered",
  ],
};

// Preset profiles that combine toolsets
export const PROFILES: Record<string, string[]> = {
  // Default — casual daily use (~26 tools)
  default: ["core", "lineup", "waivers", "trades"],

  // Full season management (~44 tools)
  full: ["core", "lineup", "waivers", "trades", "strategy", "workflows"],

  // Draft day (~18 tools)
  "draft-day": ["core", "draft"],

  // Analysis mode (~30 tools)
  analysis: ["core", "strategy", "intel", "prospects"],

  // Automation mode (~20 tools)
  automation: ["core", "lineup", "waivers", "workflows"],

  // Everything (legacy behavior)
  all: Object.keys(TOOLSETS),
};

export function resolveToolset(config: string): Set<string> {
  // Config can be: profile name, comma-separated toolset names, or "all"
  const names = config.split(",").map(s => s.trim());
  const toolNames = new Set<string>();

  for (const name of names) {
    if (PROFILES[name]) {
      for (const ts of PROFILES[name]) {
        for (const tool of TOOLSETS[ts] || []) {
          toolNames.add(tool);
        }
      }
    } else if (TOOLSETS[name]) {
      for (const tool of TOOLSETS[name]) {
        toolNames.add(tool);
      }
    }
  }

  return toolNames;
}
```

Changes to `mcp-apps/server.ts`:

Add a `toolFilter` parameter to `createServer()`:

```
export function createServer(enabledTools?: Set<string>): McpServer {
  // ... existing server creation ...

  // Pass enabledTools to each register function
  registerRosterTools(server, DIST_DIR, WRITES_ENABLED, enabledTools);
  registerStandingsTools(server, DIST_DIR, enabledTools);
  // ... etc for all register functions
}
```

Changes to each `register*Tools` function:

Wrap each `registerAppTool` call with a filter check:

```
export function registerRosterTools(
  server: McpServer,
  distDir: string,
  writesEnabled: boolean = false,
  enabledTools?: Set<string>
) {
  // Helper to skip tools not in the enabled set
  const shouldRegister = (name: string) =>
    !enabledTools || enabledTools.has(name);

  if (shouldRegister("yahoo_roster")) {
    registerAppTool(server, "yahoo_roster", { /* ... */ });
  }

  // ... repeat for each tool
}
```

Changes to `.env.example`:

```
# Tool profile: default, full, draft-day, analysis, automation, all
# Or comma-separated toolset names: core,lineup,waivers,trades,intel
# Default loads ~26 tools (core + lineup + waivers + trades)
MCP_TOOLSET=default
```

Changes to `mcp-apps/main.ts`:

```
import { resolveToolset } from "./src/toolsets.js";
```

```
// In both stdio and HTTP paths:
const toolsetConfig = process.env.MCP_TOOLSET || "default";
const enabledTools = toolsetConfig === "all"
  ? undefined  // undefined = register everything
  : resolveToolset(toolsetConfig);

const server = createServer(enabledTools);
```

AC:

- Running with `MCP_TOOLSET=default` registers ≤30 tools
- Running with `MCP_TOOLSET=all` registers all 123 tools (backward compatible)
- Running with `MCP_TOOLSET=core,intel,prospects` registers only those three sets
- Cursor (40 tool limit) works with `default` profile
- `./yf tool-count` CLI command reports how many tools are active

Task 1.2: Build the layered discovery meta-tool

Priority: Critical **Files:**

- `[NEW] mcp-apps/src/tools/meta-tools.ts`
- `[MODIFY] mcp-apps/server.ts`

What to build:

Three meta-tools that let the LLM discover and use capabilities beyond the active toolset:

```
// mcp-apps/src/tools/meta-tools.ts

import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { registerAppTool } from "@modelcontextprotocol/ext-apps/server";
import { z } from "zod";
import { TOOLSETS, PROFILES } from "../toolsets.js";

// Full registry of all tool metadata (name, description, inputSchema)
// This gets populated at startup from all register*Tools functions
export const TOOL_REGISTRY: Map<string, {
  name: string;
  description: string;
  category: string;
```

```
  inputSchema?: Record<string, any>;
}> = new Map();

export function registerMetaTools(server: McpServer) {
  // 1. Discover available capabilities
  registerAppTool(
    server,
    "discover_capabilities",
    {
      description:
        "List available tool categories and their tools. " +
        "Use this when the user asks for something outside the currently loaded t
        "or when you need to find which tool handles a specific task. " +
        "Call with no arguments to see all categories, or with a category name to
      inputSchema: {
        category: z.string().optional().describe(
          "Optional category to filter: core, lineup, waivers, trades, strategy,
          "workflows, intel, prospects, draft, mlb, history, admin"
        ),
      },
      annotations: { readOnlyHint: true },
      _meta: {},
    },
    async ({ category }) => {
      if (category && TOOLSETS[category]) {
        const tools = TOOLSETS[category].map(name => {
          const meta = TOOL_REGISTRY.get(name);
          return `  ${name}: ${meta?.description || "(no description)"}`;
        });
        return {
          content: [{
            type: "text" as const,
            text: `Tools in "${category}" (${tools.length}):\n${tools.join("\n")}
          }],
        };
      }

      // List all categories with counts
      const lines = Object.entries(TOOLSETS).map(([cat, tools]) =>
        `  ${cat} (${tools.length} tools): ${tools.slice(0, 3).join(", ")}...`
      );
      return {
        content: [{
          type: "text" as const,
          text: `Available tool categories:\n${lines.join("\n")}\n\n` +
            `Profiles: ${Object.keys(PROFILES).join(", ")}\n\n` +
            `Use discover_capabilities(category="name") for tool details in a cat
```

```
        }],
      };
    },
  );

  // 2. Get detailed schema for a specific tool
  registerAppTool(
    server,
    "get_tool_details",
    {
      description:
        "Get the full description and parameter schema for a specific tool by nam
        "Use this before calling execute_tool to understand what parameters are n
      inputSchema: {
        tool_name: z.string().describe("Exact tool name, e.g. 'yahoo_statcast_his
      },
      annotations: { readOnlyHint: true },
      _meta: {},
    },
    async ({ tool_name }) => {
      const meta = TOOL_REGISTRY.get(tool_name);
      if (!meta) {
        return {
          content: [{
            type: "text" as const,
            text: `Tool "${tool_name}" not found. Use discover_capabilities to br
          }],
        };
      }
      return {
        content: [{
          type: "text" as const,
          text: `Tool: ${meta.name}\nCategory: ${meta.category}\nDescription: ${m
            `Parameters: ${JSON.stringify(meta.inputSchema || {}, null, 2)}`,
        }],
      };
    },
  );

  // 3. Execute any tool by name (proxy)
  // NOTE: This requires a reference to the server's tool handlers
  // Implementation depends on how MCP SDK exposes tool execution internally
  // Alternative: dynamically register the needed tool on-demand
}
```

Important implementation note for `execute_tool`: The MCP SDK doesn't natively

support tool proxying. Two implementation paths:

Path A (recommended): Dynamic registration. When `discover_capabilities` identifies a needed tool that isn't loaded, prompt the user to restart with a different toolset profile. Include the suggested `MCP_TOOLSET` value in the response.

Path B (advanced): Internal dispatch. Keep all tool handler functions available but only register a subset with the MCP server. The `execute_tool` meta-tool calls handlers directly. This requires refactoring tool handlers to be callable outside of MCP registration.

For Phase 1, implement **Path A**. Path B can be a Phase 4 enhancement.

AC:

- `discover_capabilities()` returns all 12 categories with tool counts
- `discover_capabilities(category="intel")` returns detailed tool list for intel
- `get_tool_details(tool_name="yahoo_statcast_history")` returns full schema
- Meta-tools are always registered regardless of toolset profile
- Response for unloaded tools includes: "This tool is in the '{category}' toolset. To enable it, set MCP_TOOLSET to include '{category}'"

Task 1.3: Improve all tool descriptions

Priority: Critical **Files:**

- `[MODIFY] mcp-apps/src/tools/roster-tools.ts`
- `[MODIFY] mcp-apps/src/tools/standings-tools.ts`
- `[MODIFY] mcp-apps/src/tools/valuations-tools.ts`
- `[MODIFY] mcp-apps/src/tools/season-tools.ts`
- `[MODIFY] mcp-apps/src/tools/history-tools.ts`
- `[MODIFY] mcp-apps/src/tools/mlb-tools.ts`
- `[MODIFY] mcp-apps/src/tools/intel-tools.ts`
- `[MODIFY] mcp-apps/src/tools/workflow-tools.ts`
- `[MODIFY] mcp-apps/src/tools/strategy-tools.ts`
- `[MODIFY] mcp-apps/src/tools/prospect-tools.ts`

What to change:

Rewrite every tool description following this template:

```
"[When to use this tool]. [What it returns]. [Key parameters]. [When to use a dif
```

Before (current):

```
description: "Show current fantasy baseball roster with positions and eligibility
```

After (improved):

```
description: "Use this to see your current fantasy roster — all active and bench
```

Apply this pattern to every tool. High-priority rewrites:

Tool	Current description	Rewrite
`yahoo_roster`	"Show current fantasy baseball roster with positions and eligibility"	"Use this to see your full fantasy roster — every player, their assigned position, eligible positions, injury status, Statcast quality tier, and hot/cold trend. Returns player IDs needed for add/drop/trade tools. Use yahoo_free_agents to find available pickups or yahoo_lineup_optimize to set your best lineup."
`yahoo_my_matchup`	"Show your detailed H2H matchup with per-category comparison for the current week"	"Use this to see how you're doing in this week's head-to-head matchup. Shows your score vs your opponent across every stat category with running totals. Use yahoo_matchup_strategy for deeper category-level advice, or yahoo_scout_opponent to analyze their roster."
`yahoo_morning_briefing`	"Comprehensive daily briefing..."	"Use this as your first tool call of the day. Returns a complete situational report: injuries on your roster, today's lineup status, live matchup scores, category strategy, league activity, opponent moves, and top waiver targets — all in one

		call. Replaces calling 7+ individual tools. Best run daily before first pitch."
`yahoo_trade_analysis`	"Evaluate a trade by player names with surplus value analysis"	"Use this when the user asks about a potential trade. Accepts player names (not IDs) and returns z-score comparison, surplus value analysis, category impact, Statcast profiles, and news context for all players involved. Returns structured data with ai_recommendation. Use yahoo_trade_pipeline for end-to-end trade search including finding partners."
`yahoo_free_agents`	"List top free agents. pos_type: B for batters, P for pitchers"	"Use this to browse the best available free agents in the league. Set pos_type='B' for batters or 'P' for pitchers. Returns players sorted by ownership % with stats. Use yahoo_waiver_recommendations instead if you want add/drop pairs tailored to your team's weak categories."
`yahoo_streaming`	"Recommend streaming pitchers..."	"Use this to find the best pitchers to stream this week. Analyzes pitcher quality (SIERA, Stuff+), matchup strength (opponent batting stats), and schedule (two-start pitchers). Returns ranked recommendations with multi-factor scores. Best called Thursday or before the upcoming week."

Also add MCP annotations to every tool:

```
annotations: {
  readOnlyHint: true,     // for read-only tools
  // OR
  readOnlyHint: false,    // for write tools (add, drop, trade, set_lineup)
  destructiveHint: true,  // for drop, reject_trade
  idempotentHint: true,   // for auto_lineup, set_lineup (safe to retry)
}
```

AC:

- Every tool has a description of at least 2 sentences

- Every tool description starts with “Use this to…” or “Use this when…”
- Every tool description mentions at least one related tool with “Use X instead when…”
- Every tool has `readOnlyHint` set correctly
- Write tools (`add` , `drop` , `swap` , `propose_trade` , `accept_trade` , `reject_trade` , `set_lineup` , `waiver_claim` , `change_team_name` , `change_team_logo`) have `readOnlyHint: false`
- Destructive tools (`drop` , `reject_trade`) have `destructiveHint: true`
- Idempotent tools (`auto_lineup` , `lineup_optimize`) have `idempotentHint: true`

Task 1.4: Add response pagination

Priority: Important **Files:**

- `[MODIFY] mcp-apps/src/tools/roster-tools.ts` (free_agents, player_list)
- `[MODIFY] mcp-apps/src/tools/valuations-tools.ts` (rankings)
- `[MODIFY] mcp-apps/src/tools/season-tools.ts` (transactions)
- `[MODIFY] scripts/api-server.py` (corresponding endpoints)

What to change:

All list-returning tools should default to returning 25 items max with pagination metadata:

```
// Add to every list tool's response:
const response = {
  content: [{ type: "text" as const, text: formattedText }],
  structuredContent: {
    type: "free-agents",
    // ... existing data ...
    _pagination: {
      returned: items.length,
      total: totalAvailable,
      offset: offset,
      has_more: offset + items.length < totalAvailable,
      next_offset: offset + items.length,
    },
  },
};
```

Add `offset` and `limit` parameters to applicable tools:

```
inputSchema: {
  pos_type: z.string().default("B"),
  limit: z.number().default(25).describe("Max results to return (default 25, max
  offset: z.number().default(0).describe("Offset for pagination"),
},
```

Tools that need pagination (list tools returning 20+ items):

- `yahoo_free_agents` — default 25, max 50
- `yahoo_player_list` — default 25, max 50
- `yahoo_rankings` — default 25, max 50
- `yahoo_transactions` — default 20, max 50
- `yahoo_waivers` — default 20, max 50
- `yahoo_all_rostered` — default 25, max 50
- `yahoo_transaction_trends` — default 20

AC:

- All list tools default to ≤25 results
- All list tools accept `limit` and `offset` parameters
- Response includes `_pagination` object with `has_more` boolean
- No single tool response exceeds 400KB (verify with `JSON.stringify(response).length`)
- LLM can say "show me more" and Claude calls with `offset=25`

Task 1.5: Fix tool naming consistency

Priority: Important **Files:**

- All tool registration files in `mcp-apps/src/tools/`

What to change:

Audit and fix inconsistent naming. Current naming has two patterns mixed:

1. `yahoo_*` — used for Yahoo Fantasy-specific tools
2. `fantasy_*` — used for data/intel tools
3. `mlb_*` — used for MLB reference tools

This is actually fine — the three prefixes map to three data domains. Keep them as-is but document the convention:

```
yahoo_*    → Yahoo Fantasy league operations (your team, your league)
fantasy_*  → Cross-source fantasy intelligence (news, prospects, trends)
mlb_*      → MLB reference data (teams, rosters, schedules, stats)
```

The one thing to fix: Remove the `yahoo_` prefix from tools that don't use the Yahoo API:

Current name	Data source	Fix
`yahoo_statcast_history`	Baseball Savant	rename to `intel_statcast_history` or keep as-is (it's overlaid on Yahoo roster data)
`yahoo_zscore_shifts`	Internal valuations	keep (it's Yahoo player context)
`yahoo_projection_disagreements`	FanGraphs	keep (projections are in Yahoo context)

Decision: Keep existing names. The `yahoo_` prefix accurately reflects that these tools operate in the context of your Yahoo league. Renaming 100+ tools would break every existing config and documentation reference. The naming is consistent enough.

AC:

- Document the naming convention in a code comment at the top of `server.ts`
- No changes to existing tool names (stability over perfection)

Phase 2: README rewrite and setup simplification

Goal: Rewrite the README for casual fantasy baseball players first, developers second. Build a setup wizard that gets users from zero to first query in under 10 minutes.

Task 2.1: Rewrite README.md — complete restructure

Priority: Critical **Files:**

- `[MODIFY] README.md`

New README structure (in exact order):

```
<p align="center">
  <img src="banner.jpg" alt="BaseClaw" width="100%" />
</p>

# BaseClaw

**Your fantasy baseball team, managed by AI.**

Ask Claude about your Yahoo Fantasy Baseball league in plain English.
Get instant advice on trades, lineups, waiver pickups, and strategy —
powered by your actual league data.

<!-- badges -->
![License: MIT](https://img.shields.io/badge/license-MIT-blue)
![MCP Compatible](https://img.shields.io/badge/MCP-compatible-green)
![Tools](https://img.shields.io/badge/tools-123-orange)
![Docker](https://img.shields.io/badge/docker-required-blue)

---

## What can you ask?

> **"Who should I start today?"**
> BaseClaw checks your roster, today's matchups, and pitcher splits
> to recommend your optimal lineup.

> **"Should I accept this trade of Soto for two mid-tier pitchers?"**
> Analyzes z-score surplus value, category impact, and Statcast data
> for every player involved.

> **"Who's the best pickup on the waiver wire at shortstop?"**
> Scans all available shortstops, scores them against your team's
> weak categories, and recommends the top add/drop swap.

> **"How does my team compare to the rest of the league?"**
> Shows your category ranks, standings position, and strengths/weaknesses
> relative to every other team.

> **"What happened in my league overnight?"**
```

> Morning briefing: injuries, trades, waiver moves, lineup issues,
> and recommendations — all in one summary.

> **"Scout my opponent this week — where can I beat them?"**
> Analyzes their roster vs yours category-by-category and recommends
> a strategic game plan.

<details>
<summary>See 30+ more things you can ask →</summary>

Daily management:
- "Set my lineup for today — bench anyone without a game"
- "Are any of my players injured?"
- "Move [Player] to my IL and activate [Player]"

Waiver wire:
- "Who are the best streaming pitchers for next week?"
- "Show me trending players on the waiver wire"
- "Find a closer I can pick up"

Trades:
- "What's a fair trade package for [Player]?"
- "Who in my league might trade me a first baseman?"
- "Propose a trade: I'll send [Player A] for [Player B]"

Strategy:
- "Which categories should I punt?"
- "What's my path to the playoffs?"
- "Show me the power rankings for my league"

Prospect & intel:
- "Which prospects are closest to being called up?"
- "What is r/fantasybaseball saying about [Player]?"
- "Is [Player] due for regression?"

Season checkpoints:
- "Give me a weekly digest of how I did"
- "What's my season pace — am I on track for playoffs?"
- "Show my monthly strategy assessment"

</details>

Quick start (10 minutes)

Prerequisites

```
- [Docker Desktop](https://www.docker.com/products/docker-desktop/)
  (or Docker Engine + Compose)
- A Yahoo Fantasy Baseball league (any active season)
- Claude Desktop, Claude Code, VS Code with Copilot, or Cursor

### 1. Install

```bash
curl -fsSL https://raw.githubusercontent.com/jweingardt12/baseclaw/main/scripts/i
```
```

The installer will:

- Clone the repo to `~/.baseclaw`
- Walk you through creating Yahoo API credentials (takes ~3 minutes)
- Build the Docker container
- Find your league and team IDs automatically
- Configure your AI client (Claude Desktop, Claude Code, etc.)

2. Ask your first question

Open Claude and ask:

> "Show me my fantasy baseball roster"

That's it. BaseClaw handles the rest.

```
<details>
<summary><strong>Manual setup (if you prefer step-by-step)</strong></summary>
```

1. Get Yahoo API credentials:

Go to developer.yahoo.com/apps/create/ and create an app:

- **Application Name:** BaseClaw (or anything)
- **Application Type:** Installed Application
- **API Permissions:** Fantasy Sports (Read/Write)

Copy the **Consumer Key** and **Consumer Secret**.

2. Clone and configure:

```
git clone https://github.com/jweingardt12/baseclaw.git ~/.baseclaw
cd ~/.baseclaw
cp .env.example .env
```

Edit `.env` and set:

```
YAHOO_CONSUMER_KEY=your_key_here
YAHOO_CONSUMER_SECRET=your_secret_here
```

3. Start the container:

```
docker compose up -d
```

4. Discover your league:

```
./yf discover
```

This triggers Yahoo OAuth (opens a browser), finds your league, and prints your `LEAGUE_ID` and `TEAM_ID`. Copy them into `.env`, then:

```
docker compose up -d
```

5. Connect to your AI client:

```
</details>
```

Connect to your AI client

```
<details open>
<summary><strong>Claude Desktop</strong></summary>
```

Add to your config file:

macOS: `~/Library/Application Support/Claude/claude_desktop_config.json` **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "baseclaw": {
```

```
      "command": "docker",
      "args": ["exec", "-i", "baseclaw", "node",
               "/app/mcp-apps/dist/main.js", "--stdio"]
    }
  }
}


</details>


<details>
<summary><strong>Claude Code (~/.mcp.json)</strong></summary>


{
  "mcpServers": {
    "baseclaw": {
      "command": "docker",
      "args": ["exec", "-i", "baseclaw", "node",
               "/app/mcp-apps/dist/main.js", "--stdio"]
    }
  }
}


</details>


<details>
<summary><strong>VS Code (Copilot)</strong></summary>
```

Add to `.vscode/mcp.json` in your workspace:

```
{
  "servers": {
    "baseclaw": {
      "command": "docker",
      "args": ["exec", "-i", "baseclaw", "node",
               "/app/mcp-apps/dist/main.js", "--stdio"]
    }
  }
}


</details>
```

```
<details>
<summary><strong>Cursor</strong></summary>
```

Go to **Settings → MCP Servers → Add Server** and enter:

- **Name:** baseclaw
- **Command:** docker
- **Args:** exec -i baseclaw node /app/mcp-apps/dist/main.js –stdio

Note: Cursor has a 40-tool limit. BaseClaw defaults to ~26 tools, which fits comfortably. To adjust, set `MCP_TOOLSET` in your `.env`.

```
</details>
```

```
<details>
<summary><strong>Cline</strong></summary>
```

Add to Cline MCP settings:

```
{
  "mcpServers": {
    "baseclaw": {
      "command": "docker",
      "args": ["exec", "-i", "baseclaw", "node",
               "/app/mcp-apps/dist/main.js", "--stdio"]
    }
  }
}
```

```
</details>
```

```
<details>
<summary><strong>Windsurf</strong></summary>
```

Add to your Windsurf MCP configuration:

```
{
  "mcpServers": {
    "baseclaw": {
      "command": "docker",
```

```
      "args": ["exec", "-i", "baseclaw", "node",
               "/app/mcp-apps/dist/main.js", "--stdio"]
    }
  }
}


</details>


<details>
<summary><strong>Claude.ai (remote access)</strong></summary>
```

Claude.ai needs HTTPS access to the server.

1. Set `MCP_SERVER_URL` and `MCP_AUTH_PASSWORD` in `.env`
2. Put a reverse proxy in front of port 4951 (Caddy, nginx, Cloudflare Tunnel, or Tailscale Funnel)
3. Rebuild: `docker compose up -d`
4. In Claude.ai: Settings → Integrations → Add MCP Server → enter `https://your-domain.com/mcp`

The server implements OAuth 2.1 — no third-party auth needed. Access tokens last 7 days with 30-day refresh tokens.

```
</details>
```

Automate with OpenClaw

Connect BaseClaw to OpenClaw and your team manages itself. Lineups get set every morning, injuries get monitored, opponents get scouted, and waiver pickups get found automatically.

```
~/.baseclaw/scripts/setup-openclaw.sh
```

What the agent does on autopilot:

When	What	How
Every morning	Full situation report +	`yahoo_morning_briefing` +

9am	optimize lineup	`yahoo_auto_lineup`
Before lineup lock 10:30am	Weather risks + late scratches	`yahoo_game_day_manager`
Monday 8am	Scout this week's opponent	`yahoo_league_landscape` + `yahoo_matchup_strategy`
Tuesday 8pm	Rank waiver claims + submit FAAB bids	`yahoo_waiver_deadline_prep`
Thursday 9am	Find streaming pitchers	`yahoo_streaming`
Saturday 9am	Audit roster for problems	`yahoo_roster_health_check`
Sunday 9pm	Weekly recap + standings update	`yahoo_weekly_digest`
1st of month	Season checkpoint + playoff path	`yahoo_season_checkpoint`

Autonomy levels (set `AGENT_AUTONOMY` in `.env`):

Level	Behavior
`semi-auto` (default)	Sets lineups and manages IL automatically. Recommends trades, waivers, and FAAB bids for your approval.
`full-auto`	Executes all recommendations automatically and reports what it did.
`manual`	Never makes moves. Only reports recommendations.

```
<details>
<summary><strong>Manual OpenClaw setup</strong></summary>
```

1. Register the MCP server:

   ```
   npx mcporter config add baseclaw --url http://localhost:4951/mcp
   ```

2. Install skill files:

   ```
   mkdir -p ~/.openclaw/workspace/skills/baseclaw
   cp SKILL.md AGENTS.md ~/.openclaw/workspace/skills/baseclaw/
   ```

3. Register cron jobs — tell your agent: “Set up BaseClaw scheduled jobs”

```
</details>
```

How it works

BaseClaw connects to three data sources and presents them through Claude’s natural language interface:

1. **Your Yahoo Fantasy league** — real-time roster, standings, matchups, free agents, and league activity via Yahoo’s OAuth API
2. **MLB analytics** — Statcast data, pitcher Stuff+ ratings, batted ball profiles, platoon splits, and prospect intelligence from Baseball Savant, FanGraphs, and MLB Stats API
3. **Fantasy community** — trending players, news from 16 sources, Reddit buzz from r/fantasybaseball, and prospect call-up tracking

BaseClaw combines these into **z-score valuations** that measure how much each player helps your team in each scoring category. When you ask “should I trade X for Y?”, it’s not guessing — it’s comparing projected statistical impact across every category your league scores.

```
<details>
<summary><strong>Architecture details (for developers)</strong></summary>
```

[KEEP THE EXISTING ARCHITECTURE SECTION HERE — the ASCII diagram, environment variables table, project file tree, CLI commands, etc. All moved into this collapsible section.]

```
</details>
```

```
<details>
<summary><strong>All 123 tools reference</strong></summary>
```

[KEEP THE EXISTING TOOL TABLE HERE — moved into this collapsible section]

```
</details>
```

Write operations (optional)

To let Claude make roster moves (add, drop, trade, set lineup):

1. Set `ENABLE_WRITE_OPS=true` in `.env`
2. Rebuild: `docker compose up -d`
3. Set up browser session: `./yf browser-login`

Log into Yahoo in the browser that opens. Session lasts 2–4 weeks.

Uninstall

```
curl -fsSL https://raw.githubusercontent.com/jweingardt12/baseclaw/main/scripts/i
```

```
**AC:**

- README opens with value proposition and example prompts (no architecture)
- "Quick start" section is within first 2 screenfuls of the README
- 6+ AI client configuration blocks with copy-paste JSON
- Cursor-specific note about 40-tool limit included
- Architecture, tool reference, CLI commands, env vars are all in `<details>` blo
- A non-technical fantasy baseball player can understand the first 3 sections
- Total visible (non-collapsed) README is under 200 lines

---

### Task 2.2: Build interactive setup wizard

**Priority:** Critical
**Files:**

- `[MODIFY] scripts/install.sh`

**What to change:**

The existing `install.sh` already has a decent interactive flow. Enhance it with

**A. Add step-by-step Yahoo credential instructions inline:**

After prompting for `YAHOO_CONSUMER_KEY`, display:
```

```bash
echo ""
echo "  ┌──────────────────────────────────────────────────┐"
echo "  │  To get your Yahoo API credentials:              │"
echo "  │                                                  │"
echo "  │  1. Go to: developer.yahoo.com/apps/create/      │"
echo "  │  2. Click 'Create an App'                        │"
echo "  │  3. Name: BaseClaw (or anything)                 │"
echo "  │  4. Type: Installed Application                  │"
echo "  │  5. Permissions: Fantasy Sports (Read/Write)     │"
echo "  │  6. Copy the Consumer Key and Consumer Secret    │"
echo "  └──────────────────────────────────────────────────┘"
echo ""
```

B. Add `open` command to launch Yahoo Developer Console:

```
if [[ "$OSTYPE" == "darwin"* ]]; then
  open "https://developer.yahoo.com/apps/create/" 2>/dev/null || true
elif command -v xdg-open >/dev/null 2>&1; then
  xdg-open "https://developer.yahoo.com/apps/create/" 2>/dev/null || true
fi
```

C. Add a setup completion test:

After configuration, verify connectivity:

```
info "Testing connection to Yahoo Fantasy..."
if docker exec baseclaw python3 -c "
from yahoo_fantasy_api import Game
import json
with open('/app/config/yahoo_oauth.json') as f:
    creds = json.load(f)
print('OK')
" 2>/dev/null; then
  ok "Yahoo connection verified!"
else
  warn "Could not verify Yahoo connection. You may need to run: ./yf discover"
fi
```

D. Add elapsed time tracking:

```
START_TIME=$(date +%s)
# ... at the end ...
ELAPSED=$(( $(date +%s) - START_TIME ))
echo "  Setup completed in $(( ELAPSED / 60 ))m $(( ELAPSED % 60 ))s"
```

E. Auto-detect installed AI clients:

```
# Check for Claude Desktop
CLAUDE_DESKTOP_CFG=""
if [[ "$OSTYPE" == "darwin"* ]]; then
  CLAUDE_DESKTOP_CFG="$HOME/Library/Application Support/Claude/claude_desktop_con
elif [[ "$OSTYPE" == "linux"* ]]; then
  CLAUDE_DESKTOP_CFG="$HOME/.config/Claude/claude_desktop_config.json"
fi

# Check for Claude Code
CLAUDE_CODE_CFG="$HOME/.mcp.json"

# Check for Cursor
CURSOR_CFG=""
if [[ "$OSTYPE" == "darwin"* ]]; then
  CURSOR_CFG="$HOME/Library/Application Support/Cursor/User/globalStorage/cursor.
fi

# Check for VS Code
VSCODE_CFG=""
if [[ "$OSTYPE" == "darwin"* ]]; then
  VSCODE_CFG="$HOME/Library/Application Support/Code/User/globalStorage/github.co
fi

# Offer to configure each detected client
for CLIENT_NAME CLIENT_CFG in \
  "Claude Desktop" "$CLAUDE_DESKTOP_CFG" \
  "Claude Code" "$CLAUDE_CODE_CFG" \
  "Cursor" "$CURSOR_CFG" \
  "VS Code" "$VSCODE_CFG"; do
  if [ -n "$CLIENT_CFG" ]; then
    # Check if parent directory exists (client is installed)
    if [ -d "$(dirname "$CLIENT_CFG")" ] || [ -f "$CLIENT_CFG" ]; then
      read -rp "Configure $CLIENT_NAME? [Y/n] " CONFIGURE
      CONFIGURE="${CONFIGURE:-Y}"
      if [[ "$CONFIGURE" =~ ^[Yy] ]]; then
        manage_mcp_config "add" "$CLIENT_CFG"
        ok "$CLIENT_NAME configured"
      fi
    fi
  fi
done
```

AC:

- Running `install.sh` on a fresh machine with Docker installed completes in under 10 minutes
- Yahoo Developer Console link is opened automatically in browser
- Setup prints elapsed time at completion
- All detected AI clients are offered for configuration
- Connection to Yahoo is verified before declaring success
- If any step fails, the error message includes what to do next

---

Task 2.3: Create demo GIF

Priority: Important **Files:**

- `[NEW] docs/demo.gif`
- `[MODIFY] README.md` (add GIF embed)

What to create:

A 15–20 second GIF showing:

1. User opens Claude Desktop
2. Types: “Show me my fantasy baseball roster”
3. BaseClaw inline UI renders with the roster view
4. User types: “Who should I pick up to help my batting average?”
5. Claude responds with specific waiver wire recommendation

How to record:

```
# Use a screen recording tool
# macOS: built-in screen recording (Cmd+Shift+5)
# Convert to GIF:
ffmpeg -i demo.mov -vf "fps=10,scale=800:-1" -gifflags +transdiff docs/demo.gif
# Or use gifski for better quality:
gifski --fps 10 --width 800 -o docs/demo.gif demo.mov
```

Embed in README:

```
<p align="center">
```

```
  <img src="docs/demo.gif" alt="BaseClaw demo" width="700" />
</p>
```

Place immediately after the badges, before “What can you ask?”

AC:

- GIF is under 10MB
- GIF shows at least one inline UI rendering
- GIF is embedded in README between badges and “What can you ask?”
- GIF loops seamlessly

Phase 3: OpenClaw integration

Goal: Build complete OpenClaw workspace files that make BaseClaw a first-class OpenClaw skill.

Task 3.1: Create SOUL.md persona file

Priority: Important **Files:**

- `[NEW] openclaw/SOUL.md`

Content:

```
# BaseClaw Agent Persona

You are an expert fantasy baseball analyst who combines sabermetric
knowledge with practical fantasy strategy. You're like a knowledgeable
friend who happens to be a stats nerd — enthusiastic about the game,
data-driven in your analysis, but always practical about what actually
wins fantasy leagues.

## Personality

- **Conversational and direct** — explain decisions in plain language,
  not stat-speak. When you do use stats, briefly explain why they matter.
- **Proactive** — don't wait to be asked. If you see an injured player
  in an active slot, flag it. If a trending free agent fits the team's
  needs, mention it.
- **Honest about uncertainty** — projections are educated guesses.
```

```
  Say "this looks like a good bet because..." not "this will definitely..."
- **Context-aware** — remember it's a game people play for fun. Don't
  over-optimize away the enjoyment. If someone wants to roster their
  favorite player, that's valid.

## Hard limits

- **Never auto-execute roster moves without confirmation** unless
  the autonomy level is set to full-auto AND the move is classified
  as safe (lineup optimization, IL activation).
- **Never guarantee outcomes** — all analysis is probabilistic.
- **Always respect league-specific scoring rules** — check league
  context before giving generic advice. A points league and a
  categories league require totally different strategies.
- **Never trade with rivals** — don't help teams within 2 standings
  positions unless the trade clearly favors the user.

## Domain knowledge

You know: position abbreviations (C, 1B, 2B, SS, 3B, OF, Util, SP,
RP, BN, IL, IL+, NA), scoring categories (standard 5x5: R, HR, RBI,
SB, AVG / W, SV, K, ERA, WHIP), waiver priority mechanics, FAAB
bidding strategy, trade deadline implications, IL stint rules, and
minor league option rules.

## Strategy principles

- Target categories you can realistically improve — don't chase lost causes
- Stream pitchers against weak-hitting teams for counting stats
- Buy low on players with strong underlying metrics (Statcast) but
  poor surface stats
- Sell high on players with regression flags before the market corrects
- In H2H, plan your week around the matchup — sometimes punting a
  category is the right move
```

AC:

- File exists at `openclaw/SOUL.md`
- Persona is clearly defined with personality, limits, and domain knowledge
- No technical jargon about MCP, tools, or APIs — this is the agent's identity

---

Task 3.2: Rewrite AGENTS.md for OpenClaw conventions

Priority: Important **Files:**

- [MODIFY] AGENTS.md

Restructure to follow the emerging AGENTS.md convention (used by 60,000+ repos):

```
# BaseClaw Agent Instructions

## Commands

### Daily operations
- `yahoo_morning_briefing` — Run first. Gets full situational report.
- `yahoo_auto_lineup` — Set optimal lineup. Safe to run automatically.
- `yahoo_game_day_manager` — Pre-lock weather/injury check.
- `yahoo_roster_health_check` — Audit for IL waste, bust candidates.

### Weekly operations
- `yahoo_matchup_strategy` — Analyze this week's H2H opponent.
- `yahoo_waiver_deadline_prep` — Rank waiver claims with FAAB bids.
- `yahoo_streaming` — Find streaming pitchers for the week.
- `yahoo_weekly_digest` — End-of-week recap.

### Analysis (on-demand)
- `yahoo_trade_analysis` — Evaluate a specific trade.
- `yahoo_trade_pipeline` — Search for trade partners and packages.
- `yahoo_category_check` — See category strengths/weaknesses.
- `yahoo_scout_opponent` — Deep dive on any team's roster.

### Write operations (require ENABLE_WRITE_OPS=true)
- `yahoo_add` / `yahoo_drop` / `yahoo_swap` — Roster moves.
- `yahoo_propose_trade` — Send a trade offer.
- `yahoo_set_lineup` — Move specific players to positions.
- `yahoo_waiver_claim` — Submit a waiver claim with optional FAAB bid.

## Decision tiers

### Auto-execute (safe, routine)
- Set daily lineup (bench off-day players, start active ones)
- Activate player from IL when healthy
- Move player to IL when injured

### Execute + report (moderate impact, clear upside)
- Top waiver claim (FAAB ≤ 20% of remaining budget)
- Streaming pitcher add (when dropping lowest-ranked option)

### Report + wait for approval (high impact)
- Any trade proposal
- FAAB bid > 20% of remaining budget
- Dropping a player ranked in the top 150
```

```
- Any move the user hasn't discussed

## Operational rules

1. **Always call `yahoo_league_context` once per session** to learn
   the league's scoring format, stat categories, and roster rules.
   Never assume standard 5x5 — the league might use custom categories.

2. **Check regression scores before any add/drop/trade.** Buy-low
   signals prevent panic-dropping slumping stars. Sell-high signals
   maximize trade value.

3. **Adapt strategy to season phase:**
   - Pre-season/draft: Focus on draft value and sleepers
   - Early season (weeks 1-4): Small sample, trust projections
   - Midseason (weeks 5-16): Blend projections with actual performance
   - Stretch run (weeks 17-22): Win-now moves, trade futures for present
   - Playoffs: Maximum optimization, stream aggressively

4. **FAAB budget pacing:** Spend ~60% in first half, ~40% in second half.
   Never spend more than 30% on a single player unless they're elite.

5. **Trade intelligence:**
   - Surplus value > 0 means the trade favors you
   - Category fit matters more than raw value
   - 2-for-1 trades consolidate value (good for the side getting 1)
   - Catcher scarcity premium is real
   - Never help teams within 2 standings positions of you

## Project structure
```

```
baseclaw/
├── mcp-apps/ # TypeScript MCP server (tools + UI)
│   ├── server.ts # Tool registration
│   ├── main.ts # Entry point (stdio + HTTP)
│   └── src/tools/ # 11 tool files
├── scripts/ # Python backend (Flask API + analytics)
│   ├── api-server.py # Flask API (~120 endpoints)
│   ├── valuations.py # Z-score engine
│   ├── season-manager.py # In-season strategy
│   └── intel.py # Statcast + analytics
├── .env # Configuration
└── docker-compose.yml # Container setup
```

```
## Security boundaries

- Never expose Yahoo OAuth tokens in responses
- Never share other managers' personal information
- Never manipulate league settings or commissioner tools
```

```
- Write operations require explicit ENABLE_WRITE_OPS=true
```

AC:

- AGENTS.md follows the commands → rules → structure → security pattern
- Under 200 lines total
- Decision tiers are clearly defined (auto-execute vs. wait for approval)
- Season phase awareness is included
- Can be copied directly to `~/.openclaw/workspace/skills/baseclaw/AGENTS.md`

Task 3.3: Create additional OpenClaw workspace files

Priority: Important **Files:**

- `[NEW] openclaw/HEARTBEAT.md`
- `[NEW] openclaw/USER.md`
- `[NEW] openclaw/MEMORY.md`
- `[NEW] openclaw/cron-schedule.json`

HEARTBEAT.md:

```
# BaseClaw Heartbeat

## Every 4 hours during MLB season (March-October)
- Check for newly injured players on roster
- Check for healthy players stuck on IL
- Monitor free agent trending pickups (>10% ownership increase)

## Daily checks (run with morning briefing)
- Lineup optimization needed?
- Any pending trades to review?
- Any players on IL eligible to return?

## Weekly checks (Monday morning)
- Standings position change
- Category trajectory (improving/declining)
- Trade market opportunities (sell-high candidates)

## Alerts (immediate notification)
- Player placed on IL
```

```
- Trade proposed to you
- Waiver claim processed
- Player called up from minors (if on your watchlist)
```

USER.md:

```
# User Preferences

## Communication style
- Default: concise recommendations with brief reasoning
- When asked "why": provide detailed statistical breakdown
- Always include player names and relevant stats

## Risk tolerance
- Default: moderate (semi-auto autonomy)
- Adjust based on standings position:
  - Leading league: conservative (protect lead)
  - Middle of pack: moderate (calculated risks)
  - Bottom third: aggressive (swing for upside)

## Notification preferences
- Morning briefing: always
- Injury alerts: always
- Trade proposals: always
- Waiver recommendations: daily digest
- Prospect call-ups: weekly digest
```

cron-schedule.json:

```
{
  "jobs": [
    {
      "name": "morning-briefing",
      "schedule": "0 9 * * *",
      "timezone": "America/New_York",
      "command": "Run yahoo_morning_briefing then yahoo_auto_lineup. Summarize an
      "enabled": true
    },
    {
      "name": "pre-lock-check",
      "schedule": "30 10 * * *",
      "timezone": "America/New_York",
      "command": "Run yahoo_game_day_manager. Flag any weather risks or late scra
      "enabled": true
    },
    {
```

```
      "name": "matchup-plan",
      "schedule": "0 8 * * 1",
      "timezone": "America/New_York",
      "command": "Run yahoo_league_landscape then yahoo_matchup_strategy. Provide
      "enabled": true
    },
    {
      "name": "waiver-deadline",
      "schedule": "0 20 * * 2",
      "timezone": "America/New_York",
      "command": "Run yahoo_waiver_deadline_prep. Rank top claims with FAAB bids.
      "enabled": true
    },
    {
      "name": "streaming-check",
      "schedule": "0 9 * * 4",
      "timezone": "America/New_York",
      "command": "Run yahoo_streaming. Recommend best streaming pitcher add for t
      "enabled": true
    },
    {
      "name": "roster-audit",
      "schedule": "0 9 * * 6",
      "timezone": "America/New_York",
      "command": "Run yahoo_roster_health_check. Flag any IL waste, bust candidat
      "enabled": true
    },
    {
      "name": "weekly-digest",
      "schedule": "0 21 * * 0",
      "timezone": "America/New_York",
      "command": "Run yahoo_weekly_digest. Produce a narrative recap of the week'
      "enabled": true
    },
    {
      "name": "season-checkpoint",
      "schedule": "0 10 1 * *",
      "timezone": "America/New_York",
      "command": "Run yahoo_season_checkpoint. Provide monthly strategic assessme
      "enabled": true
    }
  ]
}
```


AC:

- All files exist in `openclaw/` directory

- HEARTBEAT.md defines periodic checks at 3 frequencies
- USER.md provides sensible defaults for communication and risk
- cron-schedule.json has 8 jobs with valid cron expressions
- All times are Eastern timezone
- `setup-openclaw.sh` copies these files to `~/.openclaw/workspace/skills/baseclaw/`

Task 3.4: Update setup-openclaw.sh to copy new workspace files

Priority: Important **Files:**

- `[MODIFY] scripts/setup-openclaw.sh`

What to change:

In Step 3 (Install skill files), add the new workspace files:

```
# Step 3: Install skill files + workspace templates
info "Installing skill and workspace files..."
mkdir -p "$SKILL_DIR"

# Core files
for f in SKILL.md AGENTS.md; do
  if [ -f "$BASE_DIR/$f" ]; then
    cp "$BASE_DIR/$f" "$SKILL_DIR/$f"
  fi
done

# OpenClaw workspace files
for f in SOUL.md HEARTBEAT.md USER.md MEMORY.md cron-schedule.json; do
  if [ -f "$BASE_DIR/openclaw/$f" ]; then
    cp "$BASE_DIR/openclaw/$f" "$SKILL_DIR/$f"
  fi
done

ok "Skill and workspace files installed to $SKILL_DIR"
```

AC:

- Running `setup-openclaw.sh` copies all workspace files
- Files end up in `~/.openclaw/workspace/skills/baseclaw/`

- Script is idempotent (safe to run multiple times)

Phase 4: Polish and distribution

Goal: Add badges, competitive comparison, error handling improvements, Docker MCP Catalog submission, and documentation.

Task 4.1: Add README badges

Priority: Nice-to-have **Files:**

- `[MODIFY] README.md`

Add after the title:

```
[![License: MIT](https://img.shields.io/badge/license-MIT-blue.svg)](LICENSE)
[![MCP](https://img.shields.io/badge/MCP-1.0-green.svg)](https://modelcontextprot
[![Docker](https://img.shields.io/docker/v/jweingardt12/baseclaw?label=docker)](h
[![Tools](https://img.shields.io/badge/tools-123-orange.svg)](#all-tools-referenc
[![Yahoo Fantasy](https://img.shields.io/badge/Yahoo-Fantasy%20Baseball-7B0099.sv
```

AC:

- 4–6 badges render correctly on GitHub
- Badges include: license, MCP version, Docker version, tool count, Yahoo Fantasy

---

Task 4.2: Add competitive comparison table

Priority: Nice-to-have **Files:**

- `[MODIFY] README.md` (add inside a `<details>` block)

Content:

```
<details>
<summary><strong>How BaseClaw compares to alternatives</strong></summary>

| Feature | BaseClaw | Flaim | yahoo-fantasy-baseball-mcp | Manual (Yahoo app) |
|---------|----------|-------|---------------------------|-------------------|
| Tools | 123 | ~9 | 2-3 | N/A |
| Platforms | Yahoo | ESPN, Yahoo, Sleeper | Yahoo | Yahoo |
```

```
| Write actions | ✅ Add, drop, trade, lineup | ❌ Read-only | ❌ Read-only | ✅
| Inline UI | ✅ 9 apps, 75+ views | ❌ | ❌ | ✅ Yahoo app |
| Agent automation | ✅ 8 cron jobs | ❌ | ❌ | ❌ |
| Statcast analytics | ✅ | ❌ | ❌ | ❌ |
| Prospect tracking | ✅ | ❌ | ❌ | ❌ |
| Z-score valuations | ✅ | ❌ | ❌ | ❌ |
| Trade analysis | ✅ Surplus value | ❌ | ❌ | ❌ |
| Setup time | ~10 min | ~5 min | ~5 min | N/A |

</details>
```

AC:

- Table is factually accurate (verify competitor capabilities)
- Table is inside a collapsible `<details>` block
- Renders correctly on GitHub

Task 4.3: Implement structured error messages

Priority: Important **Files:**

- `[NEW] mcp-apps/src/api/errors.ts`
- `[MODIFY] mcp-apps/src/api/python-client.ts`

Create an error catalog:

```
// mcp-apps/src/api/errors.ts

export const ERROR_MESSAGES: Record<string, {
  title: string;
  fix: string;
}> = {
  YAHOO_AUTH_EXPIRED: {
    title: "Yahoo API token expired",
    fix: "Run `./yf discover` to refresh your OAuth token, then restart: `docker
  },
  YAHOO_AUTH_MISSING: {
    title: "Yahoo API credentials not configured",
    fix: "Set YAHOO_CONSUMER_KEY and YAHOO_CONSUMER_SECRET in your .env file. See
  },
  LEAGUE_NOT_FOUND: {
    title: "League not found",
    fix: "Run `./yf discover` to find your league ID, then update LEAGUE_ID in .e
```

```
  },
  API_UNREACHABLE: {
    title: "BaseClaw API server not responding",
    fix: "Check if the Docker container is running: `docker compose ps`. Restart
  },
  BROWSER_SESSION_EXPIRED: {
    title: "Browser session expired (needed for write operations)",
    fix: "Run `./yf browser-login` to refresh the Yahoo browser session",
  },
  WRITE_OPS_DISABLED: {
    title: "Write operations are disabled",
    fix: "Set ENABLE_WRITE_OPS=true in .env and restart: `docker compose up -d`",
  },
  PLAYER_NOT_FOUND: {
    title: "Player not found",
    fix: "Check the player name spelling. Use yahoo_search to find players by nam
  },
  RATE_LIMITED: {
    title: "Yahoo API rate limit hit",
    fix: "Wait 60 seconds and try again. Yahoo limits API calls to prevent abuse.
  },
};

export function formatError(code: string, rawError?: string): string {
  const known = ERROR_MESSAGES[code];
  if (known) {
    return `❌ ${known.title}\n\n→ Fix: ${known.fix}`;
  }
  return `❌ Unexpected error${rawError ? ": " + rawError : ""}\n\n→ Check logs:
}
```

Update `python-client.ts` to detect and format errors:

```
// In the apiGet/apiPost functions, add error detection:
if (response.status === 401) {
  throw new Error(formatError("YAHOO_AUTH_EXPIRED"));
}
if (response.status === 404 && url.includes("/api/league")) {
  throw new Error(formatError("LEAGUE_NOT_FOUND"));
}
if (response.status === 429) {
  throw new Error(formatError("RATE_LIMITED"));
}
```

AC:

- All known error states return user-friendly messages with fix instructions
- No stack traces or raw Python errors reach the user
- Every error message includes a concrete next step
- Error messages work for non-technical users

Task 4.4: Create architecture documentation

Priority: Nice-to-have **Files:**

- `[NEW] docs/architecture.md`

Content should cover:

1. **Why TypeScript + Python?** — TypeScript has the strongest MCP SDK support and the only production OAuth 2.1 implementation. Python has the best baseball data libraries (pybaseball, yahoo_fantasy_api, MLB-StatsAPI). The dual-language approach plays to each ecosystem's strengths.
2. **Data flow diagram** — User → Claude → MCP SDK → BaseClaw TS (tool handler) → Python Flask API → Yahoo/MLB/Savant → Response → TS (format) → Claude → User
3. **Caching strategy** — What's cached, TTL per data source, cache invalidation
4. **Z-score valuation methodology** — FVARz method, volume weighting, park factors, projection blending
5. **Security model** — OAuth 2.1 flow, token storage, browser session isolation

AC:

- File exists at `docs/architecture.md`
- Covers the 5 topics listed
- Includes at least one diagram (ASCII or Mermaid)
- New contributors can understand the system from this document

---

Task 4.5: Submit to Docker MCP Catalog

Priority: Nice-to-have **Files:**

- `[NEW] mcp-registry.yaml` (or follow Docker's required format)

What to do:

1. Check current submission format at `github.com/docker/mcp-registry`
2. Create the registry entry with: name, description, categories, Docker image reference, environment variable requirements
3. Submit PR to the registry

AC:

- BaseClaw appears in Docker MCP Catalog search
- One-click install works from Docker Desktop
- Environment variable prompts guide Yahoo OAuth setup

Task 4.6: Implement response size guardrails

Priority: Important **Files:**

- `[NEW] mcp-apps/src/api/response-guard.ts`
- `[MODIFY] mcp-apps/src/tools/roster-tools.ts` (and all other tool files)

What to build:

```
// mcp-apps/src/api/response-guard.ts

const MAX_RESPONSE_BYTES = 400_000; // 400KB max per Block's recommendation

export function guardResponse(
  content: Array<{ type: string; text: string }>,
  structuredContent: Record<string, any>
): { content: Array<{ type: string; text: string }>; structuredContent: Record<st
  const size = JSON.stringify({ content, structuredContent }).length;

  if (size <= MAX_RESPONSE_BYTES) {
    return { content, structuredContent };
  }

  // Truncate structured content, keep text summary
  const truncated = {
    ...structuredContent,
    _truncated: true,
    _original_size: size,
    _message: `Response truncated from ${(size / 1024).toFixed(0)}KB to fit conte
```

```
  };

  // Remove the largest array fields first
  for (const key of Object.keys(truncated)) {
    if (Array.isArray(truncated[key]) && truncated[key].length > 25) {
      truncated[key] = truncated[key].slice(0, 25);
      truncated[key + "_truncated_from"] = structuredContent[key].length;
    }
    if (JSON.stringify(truncated).length <= MAX_RESPONSE_BYTES) break;
  }

  return { content, structuredContent: truncated };
}
```

AC:

- No tool response exceeds 400KB
- Truncated responses include a `_truncated: true` flag
- Truncated responses suggest using pagination
- Text content (for LLM reasoning) is never truncated — only structured/UI data

---

Appendix A: Files created or modified

New files

Path	Phase	Task	Purpose
`mcp-apps/src/toolsets.ts`	1	1.1	Toolset definitions and profiles
`mcp-apps/src/tools/meta-tools.ts`	1	1.2	Layered discovery meta-tools
`mcp-apps/src/api/errors.ts`	4	4.3	Error message catalog
`mcp-apps/src/api/response-guard.ts`	4	4.6	Response size guardrails
`openclaw/SOUL.md`	3	3.1	Agent persona
`openclaw/HEARTBEAT.md`	3	3.3	Periodic check definitions

`openclaw/USER.md`	3	3.3	User preference defaults
`openclaw/MEMORY.md`	3	3.3	Session memory template
`openclaw/cron-schedule.json`	3	3.3	Cron job definitions
`docs/architecture.md`	4	4.4	Architecture documentation
`docs/demo.gif`	2	2.3	Demo GIF for README

Modified files

Path	Phase	Task	Change
`mcp-apps/server.ts`	1	1.1, 1.2	Add toolset filtering, register meta-tools
`mcp-apps/main.ts`	1	1.1	Read MCP_TOOLSET env var
`.env.example`	1	1.1	Add MCP_TOOLSET variable
`mcp-apps/src/tools/roster-tools.ts`	1	1.1, 1.3, 1.4	Tool filtering, descriptions, pagination
`mcp-apps/src/tools/standings-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions
`mcp-apps/src/tools/valuations-tools.ts`	1	1.1, 1.3, 1.4	Tool filtering, descriptions, pagination
`mcp-apps/src/tools/season-tools.ts`	1	1.1, 1.3, 1.4	Tool filtering, descriptions, pagination
`mcp-apps/src/tools/history-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions
`mcp-apps/src/tools/mlb-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions
`mcp-apps/src/tools/intel-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions
`mcp-apps/src/tools/workflow-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions

`mcp-apps/src/tools/strategy-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions
`mcp-apps/src/tools/prospect-tools.ts`	1	1.1, 1.3	Tool filtering, descriptions
`mcp-apps/src/api/python-client.ts`	4	4.3	Error detection and formatting
`README.md`	2	2.1, 2.3, 4.1, 4.2	Complete rewrite
`AGENTS.md`	3	3.2	Restructure for OpenClaw conventions
`scripts/install.sh`	2	2.2	Enhanced wizard with inline instructions
`scripts/setup-openclaw.sh`	3	3.4	Copy new workspace files
`scripts/api-server.py`	1	1.4	Add pagination to list endpoints

Appendix B: Validation checklist

Run these checks after each phase:

After Phase 1

- `MCP_TOOLSET=default` → server reports ≤30 tools registered
- `MCP_TOOLSET=all` → server reports all 123 tools (backward compat)
- `discover_capabilities()` returns 12 categories
- Cursor connects successfully with default toolset
- Every tool has ≥2-sentence description
- Every write tool has `readOnlyHint: false`
- List tools return ≤25 items by default
- No response exceeds 400KB

After Phase 2

- README first visible section is value proposition + example prompts
- “Quick start” appears within first 2 screenfuls
- 6+ AI client configs present with copy-paste JSON
- Architecture details are in `<details>` blocks
- `install.sh` completes in <10 minutes on fresh machine
- Setup wizard auto-opens Yahoo Developer Console
- Setup wizard detects and offers to configure installed AI clients
- Demo GIF is embedded and under 10MB

After Phase 3

- `openclaw/` directory contains: SOUL.md, HEARTBEAT.md, USER.md, MEMORY.md, cron-schedule.json
- AGENTS.md follows commands → rules → structure → security pattern
- AGENTS.md is under 200 lines
- `setup-openclaw.sh` copies all workspace files to correct location
- Cron schedule has 8 jobs with valid cron expressions

After Phase 4

- README badges render on GitHub
- Error messages include fix instructions (no stack traces)
- `docs/architecture.md` exists with system diagram
- Response guardrail prevents >400KB responses

Appendix C: Key research sources informing this spec

- **Block Engineering Blog:** “Build MCP Tools Like Ogres… With Layers” — layered tool pattern (3 tools handling 200+ endpoints)
- **Block Engineering Blog:** “Block’s Playbook for Designing MCP Servers” — tool description quality, error handling, response sizing

- **GitHub Blog:** “How we’re making GitHub Copilot smarter with fewer tools” — reducing from 40 to 13 tools improved coverage from 69% to 94.5%
- **Phil Schmid:** “MCP is Not the Problem, It’s your Server” — 5–15 tools optimal, API-wrapper anti-pattern
- **MCP Discussion #1251:** Community consensus on tool count limits (30 threshold, Cursor 40 limit)
- **GitHub MCP Server:** Configurable toolsets pattern with `--toolsets` flag
- **Docker MCP Catalog:** One-click installation and OAuth handling
- **Microsoft Playwright MCP:** 12+ client-specific configuration blocks as README gold standard
- **OpenClaw documentation:** Workspace file conventions (SOUL.md, AGENTS.md, HEARTBEAT.md)
- **GitHub Blog:** “How to write a great agents.md” — commands, structure, conventions pattern